Franken FDK AAC — A Manual for the Audio Engineer

A complete guide to encoding audio in AAC / MPEG-4 and to all the options of this encoder. Written for someone who knows audio and is fluent with a computer, but who need not be a programmer or a mathematician.

Document version: July 2026. Encoder: libfdk-aac 2.0.3 + the nu774 frontend, with "Frankenstein" extensions.

Author: Michał Dziwisz. Subject-matter consultant: Patryk Faliszewski. Built on open source: libfdk-aac (Fraunhofer IIS) and the nu774/fdkaac frontend.

Part I. How AAC Actually Works

1. Why anyone needs a lossy codec

A PCM recording (what's on a CD or in a WAV file) stores every sample of sound exactly. At 44.1 kHz and 16 bits per channel in stereo, that's about 1411 kilobits per second. That's a lot. A lossy codec, such as AAC or MP3, can render the same material at 128, 96, or even 64 kilobits in such a way that the ear, under everyday conditions, hears no difference — or hears a very small one.

The trick is not about "squeezing" the sound like a ZIP file. It's about THROWING AWAY what you won't hear anyway, and storing only the rest. All the intelligence of a codec lies in guessing well what is irrelevant. That is the psychoacoustic model — a mathematical description of how human hearing works.

2. The three pillars of psychoacoustics

The encoder makes decisions based on a few phenomena worth understanding, because half the options in this program control exactly these.

SIMULTANEOUS MASKING. A loud sound at some frequency makes you stop hearing quieter sounds right next to it in the spectrum. If a loud 1000 Hz is playing, a quiet 1100 Hz signal is "masked" — the ear won't catch it. The encoder can therefore store that masked fragment very carelessly (few bits) or not at all. This is the most important bit-saving mechanism in AAC.

TEMPORAL MASKING. Just before and just after a loud hit (e.g. a snare), the ear is "deafened" and doesn't hear quiet details. The encoder exploits this.

THE THRESHOLD OF HEARING (ATH, Absolute Threshold of Hearing). There is a limit of silence below which the ear simply doesn't register sound — and this limit differs for different frequencies (we hear 2–5 kHz best, the edges of the band worse). Anything below that threshold can be thrown away.

When you hear that a codec "took away the air" or "flattened the cymbals," it almost always means that the psychoacoustic model deemed something inaudible, and your ear disagreed. A large part of this manual is learning how to shift those decisions toward your ear.

3. Bands, MDCT, and scale factors

AAC doesn't work on samples in time, but on the SPECTRUM. The signal is cut into frames (1024 samples) and transformed into the frequency domain (the MDCT transform). The resulting coefficients are grouped into SCALE FACTOR BANDS (scale factor bands, SFB for short). SFB numbering goes FROM THE BOTTOM: SFB 0 is the lowest frequencies (bass), and the higher the number, the higher up in the spectrum.

This matters, because many options in this encoder operate precisely on SFB band numbers. When we say "MS on the 5 highest bands," we mean the bands with the highest numbers.

In each band the encoder decides how precisely (how many bits) to store the coefficients. The fewer the bits, the larger the quantization error — audible as noise or a "hole" in the spectrum. The psychoacoustic model tells it where that error will hide under masking, and where it will be audible.

4. The AAC families: LC, HE-AAC, HE-AAC v2

AAC is not a single format, but a family of profiles. In this encoder you select them with the `-p` switch:

AAC-LC (Low Complexity, profile 2). The basic, most commonly used variant. It encodes the full band traditionally. Sensible from around 96 kbps stereo up. A safe choice for good-quality music.

HE-AAC (High Efficiency, profile 5, also called AAC+ or aacPlus). To LC it adds SBR — Spectral Band Replication. The AAC core encodes only the lower part of the band (e.g. up to 8–13 kHz), and the top is "recreated artificially" by SBR from the lower part and a small description. A huge bit saving while preserving the impression of a wide band. Sensible around 32–80 kbps stereo.

HE-AAC v2 (profile 29). To HE-AAC it adds PS — Parametric Stereo. Instead of encoding two channels, it encodes one (mono) plus a handful of parameters describing how to recreate stereo from it. Sensible at very low bitrates, 16–48 kbps stereo.

A practical rule: the lower the bitrate, the more "prostheses" (SBR, PS). At 320 kbps these prostheses only get in the way — use LC then.

5. What SBR is (the artificial upper band)

It's worth understanding SBR, because many options control it. SBR does NOT encode the upper band sample by sample. Instead it sends ENVELOPES — the information "in this piece of time and in this frequency range the energy has such and such a shape" — plus hints about which fragments of the lower band to "copy" upward, and how much noise to add. The decoder reconstructs the top from that.

That's why the upper band in HE-AAC sounds different from the original: it's an intelligent reconstruction, not a copy. The SBR options in this encoder let you control the density of the envelopes in time, their frequency resolution, the amount of injected noise, and so-called inverse filtering (more on that later).

6. What Parametric Stereo (PS) is

PS goes even further than SBR in saving. It sends one channel of sound and parameters describing the differences between left and right:

- IID (Inter-channel Intensity Difference) — the loudness difference between channels.
- ICC (Inter-channel Coherence) — how similar/coherent the channels are to each other.
- IPD/OPD (phase differences) — these are NOT supported in the FDK encoder (always zero); more on that honestly in the section on PS options.

From that one channel and those parameters the decoder "spreads" the sound back into stereo. At 24 kbps this is the only way for stereo to make any sense at all. At higher bitrates PS usually gets in the way and isn't used.

Part II. Bitrate, the Bit Reservoir, and Quality Control

7. CBR, VBR, and what really happens inside

CBR (Constant Bitrate) means a fixed bitrate: every second of sound takes up more or less the same amount of space. Convenient for streaming, predictable. You select it with the `-b` switch (e.g. `-b 128000` = 128 kbit/s).

VBR (Variable Bitrate) means a variable bitrate: difficult fragments (lots of detail, transients) get more bits, quiet and simple ones fewer. Theoretically

better quality per bit. In FDK, VBR is selected with the `-m` switch (modes 1–5), but — and this must be said outright — VBR in FDK is weak. The presets are conservative and don't give the freedom that, say, LAME does in MP3.

The key thing to understand: EVEN in CBR mode the bitrate locally "breathes." The BIT RESERVOIR (bit reservoir) serves this. It's a buffer: when a frame is easy, the encoder stores it sparingly and puts the saved bits into the reservoir; when a hard frame comes, it borrows from the reservoir and spends more than would follow from the average. The average is preserved, but momentarily the bitrate jumps up and down. That's why "CBR" in AAC is not perfectly flat.

This encoder exposes reservoir control to the outside, and that is the basis of our own, better VBR — see the next chapter.

8. Recipe: a VBR that doesn't cut the last frames "by force"

This is exactly the situation you often mean: you have a difficult fragment (e.g. the entrance of a whole brass section or the cymbals), the encoder released a few frames well above the average — and instead of letting that quality "live," it starts choking the following frames just to get back to the average. The effect: an audible dimming right after the loud moment.

Our solution is a quasi-CVBR: we take the CBR engine (which keeps a sensible average), but we WIDEN the reservoir and the bit range so the encoder can breathe much harder and doesn't have to "repay the debt" immediately. Four switches:

`--vbr-reservoir <bits>` — the reservoir size. Larger = the encoder can hold elevated quality longer after a difficult fragment before returning to the average.

`--peak-bitrate <bps>` — the allowed momentary peak. It lets difficult frames reach high while preserving the average.

`--max-bits-frame <n>` and `--min-bits-frame <n>` — hard bit limits per frame. This is the core of "constrained VBR": you say "no fewer than X, no more than Y per frame."

`--bitres-mode <0|1|2>` — reservoir mode: 0 full, 1 reduced, 2 off (rigid bitrate frame by frame).

A PRACTICAL RECIPE for material that should "live" after difficult fragments (~128 kbps stereo, acoustic/orchestral music):

```
fdkaac-franken-x64.exe -p 2 -b 128000 --peak-bitrate 200000 --vbr-reservoir 12000 -o out.m4a in.wav
```

What it does: the average stays around 128, but when a climax comes, frames can reach 200 kbps, and the wide reservoir (12000 bits) means the

encoder does NOT immediately choke the following frames — it lets the quality drop gently. This is exactly that "don't cut by force, let it live" effect.

Want it stronger: raise `--peak-bitrate` to 256000 and `--vbr-reservoir` to 18000. Want to hold the average more tightly (a link limit): reduce the reservoir to 4000.

AN IMPORTANT LIMITATION, honestly: this still works on the CBR engine + reservoir, not on true "by content" allocation like in LAME. The swings are moderate (on the order of ±20–50%), because AAC has a hard limit of 6144 bits per channel per frame. But this "light flight" is exactly what you usually want — and it is fully controlled.

9. Extreme bitrates: very low and very high

VERY LOW. By default FDK won't let you go below a certain floor. If you really want 6 kbps HE-AAC stereo, 4 kbps HE-AAC v2 or 3 kbps LC (even just for an experiment), use the `--unlock-bitrate` flag. WATCH OUT for one detail: normally this frontend treats small numbers with `-b` as kilobits (that is, `-b 96` = 96 kbps). In `--unlock-bitrate` mode it takes `-b` LITERALLY as bits per second, so `-b 8000` is 8000 bps = 8 kbps.

```
fdkaac-franken-x64.exe -p 29 -b 6000 --unlock-bitrate -o out.m4a
in48k.wav
```

TWO IMPORTANT CAVEATS for very low bitrates:

First: where the floors come from and what the flag removes. The common belief that HE-AAC has a hard 16 kbps minimum is wrong; this manual used to repeat it too. 16 kbps is only the lowest row of the SBR tuning table for 48 kHz stereo. For mono and lower sample rates the table starts at 8..12 kbps, for HE-AAC v2 at 8 kbps. Stock FDK raises anything below to that row. With `--unlock-bitrate` the encoder takes the lowest row and keeps your bitrate: the SBR grid is the same, the AAC core simply gets fewer bits. Under LD and ELD the flag also removes the 8 kbps per channel rule and the 208-bit transport reserve (the real header overhead is still checked) and uses the narrowest bandwidth below the bandwidth table. The lowest `-b` that still encodes at 48 kHz (stock floor it used to be raised to in brackets): HE-AAC stereo 5050 (16008), HE-AAC mono 2505 (12008), HE-AAC v2 3122 (12008), LD 8291 (19500), ELD 9062 (19500), ELD with SBR about 10500..11300 (36008..60008), LC 2659 (9750). At 24 and 32 kHz the values are lower still. The real hard limit is the estimate of the SBR payload itself (about 2.5 kbps per channel plus PS): below it the core has no bits at all and the encoder stops with a clean "encoder initialization failed".

At such values expect dense passages to come out above `-b`: a frame that cannot fit even its own headers and side information overshoots, and the reservoir catches up over the next frames. Example: `-b 6000` for HE-AAC

stereo gave a real 7290 bps on music. Check the file size, not only the value in `--verbose`. Listen to such LD and ELD streams with the Fraunhofer decoder; ffmpeg 6.1 has trouble with them even from the stock encoder.

Second — WATCH the interpretation of `-b` in unlock mode. Normally this frontend treats small numbers as kilobits: `-b 128` = 128 kbps, `-b 1152` = 1152 kbps. But with `--unlock-bitrate` the number is taken LITERALLY as bits per second: `-b 8000` = 8000 bps = 8 kbps. If by mistake you wrote `-b 1152 --unlock-bitrate` (thinking "1152 kbps"), you'd get 1152 bps — practically nothing. The program will warn when a value looks like that mistake. Rule: in unlock mode always give the full number of bits (e.g. `-b 6000`, not `-b 6`).

VERY HIGH. The upper ceiling is 6144 bits per channel per frame. At 48 kHz that works out to about 576 kbps per channel; in stereo the real limit appears around 600 kbps total. If you want 1000+ kbps, you have to raise the sample rate above 48 kHz (a 96 kHz source) — then the ceiling rises and you'll reach 1024, even 1152 kbps. This is a limit of the AAC standard itself.

A note on unambiguity: at such high bitrates (1152) you do NOT use `--unlock-bitrate` (it serves for going down), so `-b 1152` there unambiguously means 1152 kbps. The interpretation collision applies only to unlock mode, where you're operating with small bps numbers anyway. In practice these two worlds don't touch.

9a. Is it worth doing an "afterburner plus" — more iterations at extreme bitrate

Since optimization at very high bitrate is your thing, one thing must be said outright, otherwise you'll waste time on a dead end. Inside the encoder there's a loop that iteratively picks the quantization (an internal parameter "maxIterations"; with the afterburner on it gives 5 tries, off 2). It's tempting to think "I'll give it 20 iterations, it'll be more precise." But this loop is a RESCUE mechanism for a bit SHORTAGE — it only kicks in when the frame doesn't fit in the budget and needs to be pressed down. At extremely HIGH bitrate the budget is enormous, the frame fits right away, so the loop almost never goes into further iterations. Raising the limit does nothing there — it's not "more polishing," but "a higher emergency-pressing limit," which you don't use anyway. The afterburner itself (`--afterburner 1`) already selects a more precise tuning table (and this is on by default) — and higher up this particular mechanism doesn't reach, because there are no additional levels in the codec.

What REALLY gives more detail at high bitrate is not the number of iterations, but LOWERING THE MASKING THRESHOLDS, so the encoder deems more things worth encoding at all: `--ath-scale` below 256, `--spread-mask` below 256, and `--side-bias` above 0 (more bits into the stereo width).

These are the levers that actually convert surplus bits into preserved detail (see chapter 18). Quantization iterations don't.

VERY HIGH (continuation of the proper optimization) — see chapter 18.

Part III. Standard Options (Not Just Frankenstein)

These are the switches available in every version of this frontend. It's worth knowing them, because they're the foundation on which you build the rest.

10. Profile selection and basics

`-p <n>` — profile (Audio Object Type). The most important: `2` = AAC-LC, `5` = HE-AAC, `29` = HE-AAC v2, `23` = AAC-LD (low delay, for communication), `39` = AAC-ELD. For music: 2 for high bitrates, 5 for medium, 29 for the lowest.

`-b <n>` — bitrate in bits per second for CBR (e.g. `-b 192000`). Small numbers the frontend multiplies ×1000 (`-b 192` = 192 kbps) — outside `--unlock-bitrate` mode.

`-m <n>` — VBR mode 1–5 (1 lowest quality, 5 highest). As mentioned, weak in FDK; for serious work use CBR + chapter 8.

`-o <file>` — output file. The `.m4a` extension gives an MP4 container; `-f 2` switches to a raw ADTS stream (`.aac`).

`-w <n>` — the bandwidth of the core in Hz. Note: under SBR this does not work as you'd expect — to control the core band under HE-AAC use `--core-cutoff` (Part IV).

11. Afterburner and signaling

`--afterburner <0|1>` — the afterburner is an additional, slower quantization optimization algorithm. `1` = on (better quality, slower), and this is the default and recommended. Turn it off only when you care about speed.

`-s <n>` / signaling — how SBR/PS is signaled in the stream (compatibility with older decoders). The default "auto" is good in most cases.

12. Container and metadata

`--moov-before-mdat` — places the MP4 index at the start of the file (useful for progressive streaming). Standard tags (title, artist, etc.) are set with the `--title`, `--artist`, `--album` and related switches.

Part IV. Frankenstein Options — Controlling the Encoder's Insides

This is the heart of this encoder: switches that in ordinary FDK are hard-wired fixed, and here exposed to the outside. Each is described practically — what it does for the ear, when to use it, and what values make sense.

13. Stereo in the core: MS and Intensity

MS STEREO (Mid/Side). Instead of encoding the left and right channels separately, it encodes the sum (mid = L+R) and the difference (side = L−R). When the channels are similar (and in music they usually are), the difference is small and cheap to store. It's almost free saving.

`--msmask <0|1>` — force: `0` = separate L/R everywhere (full channel independence), `1` = MS on all bands. By default the encoder decides for itself per band.

`--msbands <n>` — restrict MS to bands with a number smaller than n. That is, it takes the N LOWEST bands (remember: bands are numbered from the bottom, 0 = bass). Above that — pure L/R. Example: `--msbands 6` = MS only on the 6 lowest bands.

`--msbands-lo <lo>` and `--msbands-hi <hi>` — this is a "FROM–TO" pair. You give BOTH switches and define the range of band numbers in which MS is allowed; outside that range it goes pure L/R. This lets you place MS ANYWHERE, including at the very top of the spectrum — which `--msbands` (always from the bottom) can't do.

How to remember it: `--msbands` = "from the bottom up to number N"; `--msbands-lo/-hi` = "from number LO to number HI." Examples (assuming about 49 bands in LC stereo):

- MS only on the 5 HIGHEST bands (merge the noise up top, leave the bottom in full stereo): the highest bands are numbers 44–48, so `--msbands-lo 44 --msbands-hi 48`.
- MS only in the middle of the band: `--msbands-lo 10 --msbands-hi 30`.
- MS on the 6 lowest: simpler `--msbands 6` (the same as `--msbands-lo 0 --msbands-hi 5`).

How many bands you really have for a given mode and frequency is shown by `--verbose` (the "active SFBs" field) — check the top number there before you set `--msbands-hi`.

In short blocks the same numbers mean the same frequency bands, counted from 0 inside each window group (a short frame has about 14 bands at 44.1/48 kHz). So `--msbands-lo 0 --msbands-hi 8` keeps bands 0..8 in every group, and a range above the short band count leaves short frames in pure L/R.

STEERING THE STEREO BALANCE (the main knob). `--side-bias <dB>` is the switch to reach for when you want to control how much of your bit budget goes into the stereo width. It shifts the masking threshold of the SIDE channel (L−R) on the bands coded in MS, at the exact point where FDK decides whether a scale-factor band gets coded or dropped to zero (`energy > threshold` in `sf_estim.cpp`). The sign is the effect. A POSITIVE value steers MORE bits to the side channel: the threshold drops, fewer side bands are zeroed, and the ones that survive are quantized more finely — you get cleaner stereo width, reverb tails and ambience — at the mid channel's expense. A NEGATIVE value deliberately degrades the side: it raises the threshold, side bands fall out, and the image narrows toward mono. Nothing exotic is happening here; this is the very same energy-versus-threshold decision that LAME rides for its bit allocation and that MusePack drives with its `--ms` control. It is a fixed-budget tradeoff, so at low bitrate you will hear the mid channel give up some bits to feed the side — that is expected behaviour, not a fault. Sane human range: **+3 to +9** for "wider and more spacious"; go negative only when you deliberately want the extreme, artistic, low-bitrate mangling of the width. The value is in decibels (decimal), and `0` means off — bit-identical to stock FDK.

Fractions work. You can type a dot or a comma and up to three decimal places (`-0,23`, `-0.75`, `1,125`); anything else is rejected with an error. One real boundary exists: at about -0.97 dB the raised side threshold crosses the minimum-SNR line built into FDK and most side bands drop out at once. That is why -0.95 and -0.98 sound clearly different, while -1, -4 and -12 sound the same: the side is simply cut by then. For gentle narrowing use 0 to -0.95.

SHAPING THE SIDE CUTOFF. `--side-knee <dB>` controls HOW SHARPLY a side band flips between "coded" and "zeroed" at that threshold. Stock FDK is a hard cliff: the instant a band's energy drops to or below the threshold, the whole band is thrown away. A POSITIVE value gives you a SOFT knee — bands sitting up to N dB *below* the threshold are still kept (coded at the coarsest scale factor) instead of dropped, so the side fades out gradually rather than switching off abruptly; reverb and "air" decay more smoothly. A NEGATIVE value gives a HARD knee — bands that only just clear the threshold (within N dB *above* it) are forced to zero anyway, cutting the side off earlier for a leaner, more aggressive result. It is orthogonal to `--side-bias` and combines with it: bias sets *where* the threshold sits, knee sets *how soft the edge is*. Sane range **+3 to +6**. `0` = off.

TUNING THE QUIET BANDS GLOBALLY. `--mask-slope <dB>` is a global (both mid AND side) adjustment of FDK's Masking-Slope-Adaptation — a NON-masking heuristic (`adj_thr.cpp`) that relaxes the required SNR for scale-factor bands whose energy sits far below the frame's average (stock: more than about 10 dB below). In plain terms, FDK deliberately starves very quiet bands to save bits, and this knob shifts the "how far below average

before I stop caring" threshold. A POSITIVE value raises it, so fewer quiet bands get starved — more detail in quiet passages, reverb tails and decays, at the cost of bits. A NEGATIVE value lowers it, so quiet bands are starved harder — leaner and hollower, with more bits left for the loud material. It is the same family as `--side-bias`, but applied to both channels and keyed on energy-versus-average rather than the MS threshold. It is subtle on dense material (it only touches the very quietest bands) and most audible on sparse or reverberant content. Sane range **±6 to ±12**. `0` = off.

`--ms-precision <n>` — VERY EXPERIMENTAL, and these days you almost certainly want `--side-bias` instead. It scales the precision of MS bands globally (both mid and side together), on the Q8 scale, LAME `-q` style: 256 = no change, 384 ≈ 1.5×, 512 ≈ 2×. In practice its reach is limited — above roughly 600–800 the threshold hits FDK's hard floor, and under CBR the bits are merely shuffled between bands, so the sound stops changing. For stereo tuning it has been superseded by `--side-bias`/`--side-knee`, which act per-channel exactly where it matters. Kept for completeness.

`--mid-bias <n>` — also VERY EXPERIMENTAL and rarely needed. Above 256 it RAISES the threshold of the mid (L+R) channel after the MS butterfly, deliberately freeing bits for the side channel. The cleaner, better-measured way to shift the mid↔side balance is `--side-bias`, which pulls from the same budget from the side end. 256 = off.

INTENSITY STEREO (IS). A more aggressive technique: for high bands, where the ear poorly localizes direction, the encoder sends one energy envelope instead of two channels. It saves a lot, but can narrow the scene.

`--is <0|1>` — globally turn IS on/off.

`--is-aggression <0..100>` — a CONSUMER slider for IS aggressiveness. This is what you usually want to use. 0 = FDK's default behavior (very cautious), 100 = maximally aggressive intensity. Why it's needed: FDK has three independent "gates" letting IS in (the bitrate-to-band ratio, the channel-correlation threshold, the minimum number of contiguous bands) and they must be met SIMULTANEOUSLY. Moving one threshold does nothing, because another blocks it. This slider moves all of them at once. Start at 40–70 and listen.

Advanced IS thresholds (for those experimenting — they act AFTER passing through the aggressiveness slider): `--is-min-sfbs <n>` (the minimum number of contiguous bands), `--is-corr-thresh <n>` (the channel-correlation threshold, Q8), `--is-lr-ratio <n>` (the L/R ratio threshold, Q8). A practical note: these thresholds act COUNTERINTUITIVELY — a lower correlation-threshold value means MORE aggressive IS, not the other way around.

Placing IS by band. `--is-lo <sfb>` and `--is-hi <sfb>` let you RESTRICT intensity stereo to a range of bands: IS is allowed only from `--is-lo` upward, and only up to `--is-hi` inclusive. This never forces IS on — it only limits where FDK may place it. In practice IS lands on the LOW bands at low bitrate, so scan small values to actually see the effect. If you want to go further and FORCE it, `--is-force-lo <sfb>` and `--is-force-hi <sfb>` push intensity stereo onto the whole range regardless of the correlation, min-sfbs, and loudness gates. This is a laboratory mode: because IS is lossy and directional (the right channel is zeroed and only a panning coefficient survives), forcing it can deliberately wreck the stereo image. The stream still stays standard-compliant.

14. Band and cutoff (including audiophile full band)

`--core-cutoff <Hz>` — the upper limit of the band encoded by the core, in Hz. It works also under SBR (unlike `-w`). E.g. `--core-cutoff 7500` under HE-AAC v2 gives the core 7.5 kHz, and SBR does the rest.

A note on the effective cutoff in `--verbose`: the AAC spectrum is quantized into SFB boundaries, so the encoder cannot cut at an arbitrary Hz value — it snaps the cutoff to the nearest SFB boundary. `--verbose` therefore reports the REAL cutoff, which may differ from the `-w`/`--core-cutoff` value you typed (e.g. `-w 17300` shows up as `17915 Hz (SFB-anchored)`). This is not an error; it is the true band that ends up in the stream.

`--uncap-bandwidth` — audiophile. FDK has a hard-wired band limit for the core: the minimum of 20 kHz and half the sample rate. This means that EVEN with a 96 kHz source and a high bitrate, in reality nothing above 20 kHz was encoded. This flag removes the limit — `--core-cutoff` can then reach all the way up to half the sample rate. For 96 kHz material and listeners who want the full band:

```
fdkaac-franken-x64.exe -p 2 -b 320000 --uncap-bandwidth --core-cutoff
40000 -o out.m4a in96k.wav
```

Measured: without this flag there is practically zero energy above 20 kHz; with it the band up to 40 kHz is really encoded.

15. SBR — controlling the upper band

Overrides the settings from the internal SBR tuning table.

`--sbr-start <n>` / `--sbr-stop <n>` — the start and stop indices of the SBR band. NOTE: FDK validates the combination — a bad pair will give "encoder initialization failed." Legal pairs form a diagonal band and depend on bitrate and sample rate. At HE-AAC 64 kbps stereo from 48 kHz, for example, start 0..4 is illegal with every stop, while 8/9, 8/14 and 14/12 work. Try one change at a time and read `--verbose`; the reference card lists the measured values.

`--sbr-freqscale <n>` — frequency grouping (0 linear, higher = finer logarithmic). `--sbr-noise-bands <0..3>` — the density of the SBR noise description: 0 is a single noise band, 1, 2 and 3 are about 1, 2 and 3 bands per octave. The SBR header field has 2 bits, so larger values cannot be signalled. `--sbr-amp-res <0|1>` — the amplitude resolution of the envelope (0 = 1.5 dB, 1 = 3 dB).

`--sbr-num-env <1|2|4>` — the number of envelopes per frame = the SBR time resolution. More envelopes = better-rendered changes in time in the upper band, at the cost of bits. IMPORTANT: this option FORCES a static time grid (it disables the transient detector), so on material with sharp attacks it may be worse. Good for stable, ringing sounds. (The value 8 exceeds the standard grid and is rejected.)

`--sbr-freqres-fixfix <0|1>` — the frequency resolution of the envelope.

`--sbr-stereo-mode <0..3>` — the stereo mode in the SBR layer (separate from core MS!): 0 = mono, 1 = LR (full, independent separation of left and right in the upper band), 2 = coupling (economical: shared envelope + level/balance), 3 = switch (by default the encoder chooses per frame). Force 1 for maximum stereo separation up top (audiophile), 2 for bit saving at low bitrate.

`--sbr-invf <0..3>` — inverse filtering (inverse filtering). It's a mechanism controlled by a TONALITY ESTIMATOR: SBR analyzes whether the upper band should be more tonal or more noise-like, and accordingly "whitens" the copied material. 0 = off, 1 = weak, 2 = medium, 3 = strong. Stronger = less "metallic" ringing up top, at the cost of some detail. Usually the automation controls it; force manually when you hear metallicness.

Forcing has one drawback: a single level then applies to the whole file, also in the bands and moments where the automation would have chosen differently. The other way is to leave the decision to the automation and change only its rules. The decision is still made per band and per frame.

`--sbr-softness <0..7>` — one knob from "hard, metallic" to "very soft". `2` is exactly the stock music estimators, `4` the stock speech estimators (without the rest of `--speech`, so no different crossover). Levels 0..1 give less whitening, 3 keeps at least weak, 5..7 add noise and whitening. Measured from the bitstream: the mean filtering level rises from 0 to about 3, and the flatness of the decoded SBR band from 0.37 to 0.42 on speech and from 0.40 to 0.52 on tonal material. Frame-to-frame switching stays at the stock level.

Expert knobs, each wins over the macro: `--sbr-invf-min` and `--sbr-invf-max` bound the automation from below and above (e.g. `--sbr-invf-max 2` = never strong). `--sbr-invf-bias <-3..3>` shifts every decision by that many steps. `--sbr-invf-table 0|1` picks the music or speech decision table

regardless of `--speech`. `--sbr-invf-hyst` (0..2, stock 1) is the hysteresis of the borders: smaller reacts faster and switches more often. `--sbr-invf-sbr-thr` and `--sbr-invf-orig-thr` shift the tonality borders of the patched and the original band, roughly in dB. Note: shifting the borders can make the decision flicker between frames, which is why the macro does not use them. `--sbr-invf-smooth 0..2` sets how many frames the estimator averages.

Noise: `--sbr-nf-weight 0..1` is the weight of the original-versus-patch tonality gap in the noise level (music 0.25, speech 1.0), `--sbr-nf-weight-from 1..3` sets from which filtering level the bands get that weight (stock: strong only), and `--sbr-nf-smooth 0` turns off the time smoothing of the noise. One important finding from measurement: the noise is capped by `--sbr-noise-max`. With the tuning-table ceiling (0.125) about 95% of the bands already sit at it, so the weight alone often changes nothing. Pair it with `--sbr-noise-max 3` or 6.

```
fdkaac-franken-x64.exe -p 5 -b 32000 --sbr-softness 6 -o out.m4a
in.wav
fdkaac-franken-x64.exe -p 5 -b 32000 --sbr-invf-max 2 --sbr-invf-hyst
1.5 -o out.m4a in.wav
```

`--sbr-noise-floor-offset <0..11>` — the offset of the noise level injected by SBR. Larger values = more fill noise in the reconstruction. FDK accepts only 0..11; anything else is rejected with an error.

`--sbr-header-period <n>` — how many frames apart the SBR headers are written, which decides how fast the SBR upper band "kicks in" when a decoder tunes into a LIVE HE-AAC stream (Icecast/Shoutcast). Here's the catch: the whole SBR configuration lives in a periodic header, not in every frame. A listener who joins mid-stream hears only the AAC core (muffled, no top) until the next header arrives. Set `1` and a header is written in every frame, so a decoder locks onto SBR almost instantly (~23 ms) — the right choice for a stream people join at random moments. Larger values stretch that core-only stretch. FDK's default is 10 frames, about 0.46 s at 44.1 kHz (an HE-AAC frame is 2048 output samples). The value is taken literally, without the one-second ceiling stock FDK applies.

`--sbr-header-time <s>` — the same as a time in seconds, decimal (0.01 to 600, e.g. `3.5`). It wins over `--sbr-header-period`. Use it when a stream operator requires a specific gap between valid SBR headers, for example 3.5 s. It works in HE-AAC and HE-AAC v2 (in-band SBR header, any transport) and in AAC-ELD with SBR (`-p 39 -L 1`). ELD has no in-band SBR header: its SBR configuration lives only in the stream config, so the switch then sets the StreamMuxConfig repetition period of LATM/LOAS (`-f 6`, `-f 7`, `-f 10`). With ELD in ADTS or M4A the config sits only in the container and there is nothing to repeat.

`--verbose` prints the effective period in frames and milliseconds. Decoder-side measurement with `--sbr-header-time 3.5`: the longest wait for the SBR band after joining mid-stream was 3.41 s (HE-AAC), 3.46 s (HE-AAC v2) and 3.52 s (ELD).

16. Parametric Stereo (HE-AAC v2)

`--ps <0|1>` — force the sending of the IID parameter (loudness difference): 0 = never (flattens the stereo image to mono-like), 1 = always.

`--ps-iid-quant <0|1>` — the precision of IID quantization: 0 coarse, 1 fine.

`--ps-icc <0|1>` — force ICC (Inter-channel Coherence — channel coherence/similarity) on/off. `--ps-icc-mode <0|1>` — the ICC rotation mode (ROT_A / ROT_B). Be aware that the rotation mode is *signalling only*: both modes describe the same stereo matrix, the decoder just derives it differently. It is a compatibility knob, not a quality one.

PS resolution: bands (frequency) and envelopes (time)

The two knobs that genuinely change stereo quality are the ones that decide *how many* stereo parameters get sent in the first place. The MPEG-4 PS baseline allows two band resolutions and up to four parameter envelopes per frame, and stock FDK picks both purely from the bitrate — so at anything above 36 kbps you are locked into 20 bands and 4 envelopes with no way to hear the alternatives.

`--ps-bands <10|20>` — the frequency resolution of the stereo parameters. Twenty bands describe the stereo image in finer frequency detail; ten describe it more coarsely but spend fewer bits. At very low rates the coarse grid can actually sound steadier, because each band gets averaged over more spectrum and the parameters jitter less between frames.

`--ps-env <1|2|4>` — the time resolution. One envelope means the whole frame gets a single stereo snapshot; four means the panorama is sampled four times per frame. This is what decides whether a moving pan or a transient keeps its position or smears across the frame.

`--ps-env-reduce <0|1>` — and this is the one that matters most, because of a trap. Asking for four envelopes does not mean you get four. FDK runs an automatic reduction loop that keeps halving the envelope count — four to two to one — as long as neighbouring envelopes look similar according to a hardcoded error threshold. On ordinary music that threshold is met very often, so the encoder quietly falls back to one snapshot per frame. Setting `--ps-env-reduce 0` switches that loop off and makes `--ps-env` mean what it says.

The measurement is unusually clear here. On a four-second probe with a panorama deliberately swept back and forth at 0.25 Hz plus alternating

left/right transients, encoded as HE-AAC v2 at 48 kbps, comparing the panorama trajectory of the decoded result against the source: the stock encoder lands at 0.177 RMS panorama error (0.9655 correlation). Asking for four envelopes on its own changes *nothing at all* — the output is byte-for-byte identical to stock, because the reduction loop undoes the request immediately. Add `--ps-env-reduce 0` and the error drops to 0.117 (0.9944 correlation) at essentially the same file size: a third less panorama error, for free. If you take one setting from this chapter, take `--ps-env 4 --ps-env-reduce 0`.

`--ps-noenv-skip <0|1>` — a related piece of hidden behaviour. Beyond thinning out envelopes, FDK can also skip stereo parameters *entirely* for a stretch of frames: when successive IID/ICC sets look similar it may send up to ten consecutive frames carrying no stereo information at all, leaving the decoder to coast on the last values it received. On material where the image drifts slowly this is audible as the stereo picture briefly flattening and then snapping back. `--ps-noenv-skip 0` forbids it and forces parameters into every frame.

IPD: sending the phase (`--ps-ipd 1`)

Stock FDK never transmits inter-channel phase. The fields are hardcoded to zero with the comment "IPD OPD not supported right now" — but that comment overstates the distance to a working implementation. The encoder already accumulates both the real and the imaginary part of the left/right cross-spectrum for every band and envelope; it uses only the magnitude, to compute ICC, and discards the angle. That angle *is* IPD. The Huffman tables and the bitstream writer were already present too. So the missing piece was never "phase analysis from scratch" — it was taking an `atan2` of two numbers already sitting in the loop, rounding it onto the 8 defined phase steps, and switching on the writer.

Why it matters: without phase, PS places sound using level differences and correlation alone. That covers anything panned by level, but not a source positioned by *timing*. A real inter-channel delay — the way low-frequency localisation works in nature — is invisible to IID and ICC, because both channels carry the same signal at the same level and differ only in when it arrives.

The measurement uses probes built exactly that way: identical signal in both channels, equal level, offset only by a delay. Comparing the decoded phase profile against the source, within the range IPD actually covers (60-690 Hz):

Inter-channel delay	phase error, IPD off	phase error, IPD on
0.25 ms	35.0 deg	**14.9 deg**
0.4 ms	56.1 deg	**24.9 deg**

Inter-channel delay	phase error, IPD off	phase error, IPD on
0.7 ms	67.4 deg	71.8 deg

At a quarter and four tenths of a millisecond the phase error more than halves, with identical figures at 32 and 48 kbps. At 0.7 ms the benefit disappears — and that is the representation reaching its limit rather than a fault. By the top of the IPD range a 0.7 ms delay has swung through nearly the whole plus-or-minus 180 degrees, so a 45-degree grid can no longer say unambiguously where it is. This is the same phase-ambiguity ceiling that makes human hearing rely on timing cues mostly down low.

One practical note before the second parameter: IPD reaches only about the lowest 690 Hz, because its eleven parameter bands live inside the first three QMF bands. Listen or measure above that and you will find nothing at all.

OPD: the other half of the pair

IPD alone is not the whole story, and the reason is worth spelling out. IPD is the phase of the left channel relative to the right; **OPD is the phase of the left channel relative to the mono downmix.** The decoder uses both: it rotates the paths fed from the downmix by `OPD`, and the others by `OPD - IPD`. Subtract those and the `OPD` cancels, so the inter-channel *difference* always comes out as IPD no matter what OPD says — but where that rotation sits relative to the downmix is entirely OPD's business.

Leave OPD at zero and the left channel gets pinned to the downmix phase while the whole rotation is dumped onto the right one. The difference is still right, which is exactly why this is easy to miss: any measurement of `arg(L) - arg(R)` is blind to it. Measure each channel's absolute phase against the source instead, inside the IPD range, and the asymmetry is obvious — as is OPD removing it:

Inter-channel delay	OPD = 0 (L / R)	OPD computed (L / R)
0.25 ms	14.7 / 20.1 deg	14.4 / **17.3** deg
0.4 ms	23.0 / 28.0 deg	**15.5** / **17.8** deg
0.7 ms	48.1 / 56.2 deg	**40.9** / **36.3** deg

At `OPD = 0` the two channels never agree — 14.7 against 20.1, later 48.1 against 56.2. With OPD computed they converge, and the benefit grows with the delay: +6.4 degrees on average, +13 at 0.7 ms. Note also that this is the case where the *plain* IPD measurement showed nothing: at 0.7 ms the inter-channel difference was already past what a 45-degree grid can resolve, yet the absolute phase of each channel still improved substantially.

And it costs nothing to derive. The downmix is `L + R`, so

`sum(L * conj(L+R)) = sum(|L|^2) + sum(L * conj(R))`

whose real part is `pwrL + pwrCr` and whose imaginary part is `pwrCi` — the same accumulators already feeding IID, ICC and IPD. So `OPD = atan2(pwrCi, pwrL + pwrCr)`: one more `atan2` over numbers that were already sitting in the loop, no downmix signal needs to be passed in from elsewhere. For equal-level delay this reduces analytically to `IPD/2`, since `arg(1 + e^{-i.phi}) = -phi/2` — a handy sanity check, though the general form above is also correct when the channels differ in level, where `IPD/2` would not be.

An earlier revision of this encoder shipped OPD at zero and claimed measurement showed `IPD/2` did not help. That measurement was scoped to `arg(L) - arg(R)`, which as explained above cannot see OPD at all. The verdict was wrong and has been corrected; `--ps-opd 0` remains available for A/B comparison.

One practical consequence, learned the hard way and fixed in v1.3.1: the phase extension does not carry the number of phase bands. The decoder derives that from `iid_mode`, and only from a header that also signals IID. FDK decides to send IID from a loudness-difference heuristic, so on material whose channels sit at nearly equal levels — near-antiphase content above all — it would legitimately signal no IID, and the decoder would then read a different number of bits than the encoder wrote and throw the phase away. Turning on `--ps-ipd` therefore also makes the stream signal IID. It is a normal parameter, cheap and always legal, and it is the only route by which the band count reaches the decoder.

Compatibility is safe by construction. The data sits in a length-prefixed extension, so a decoder that does not implement phase synthesis — including FDK's own baseline PS decoder — reads the byte count and skips it, which the standard explicitly allows. Verified in practice: with the switch off the output is byte-identical to before, and with it on both ffmpeg and faad decode cleanly at 24, 32, 48 and 64 kbps.

SBR detectors: attacks and fabricated harmonics

Two detectors inside SBR decide most of what the replicated high band actually sounds like, and until now neither was reachable from the command line.

The **transient detector** decides whether an attack gets its own envelope or is averaged into a longer stretch. When it misses an attack, the energy of the hit gets spread backwards in time — you hear a short burst of high-frequency hiss just *before* the hi-hat, not after it. That is pre-echo, and in the SBR band it is one of the more recognisable coding artefacts.

`--sbr-tran-thr <n>` scales the master threshold that decides how easily the detector fires (value*100, so 100 is unchanged; values below 100 make it

fire more readily, values above 100 less readily). This turned out to be the single most effective knob in the group. Measured on a signal of 24 percussive attacks plus decaying tones between 8 and 13 kHz, counting high-band energy that appears before an attack but is absent from the source: stock scores 1.610 at 32 kbps, and lowering the threshold to 40 drops that to 0.041 — pre-echo essentially gone. The same thing happens at 64 kbps (1.186 to 0.059). The effect saturates below 60 and disappears entirely at 100 and above, where output is byte-identical to stock. On percussive material `--sbr-tran-thr 40` is the place to start.

`--sbr-tran-peak <n>` controls how *peaky* a candidate must be to count as an attack — the raw 0.90 constant, entered as 90. `--sbr-tran-split <n>` governs how readily a frame with no detected transient is still split into two envelopes, which buys time resolution at the cost of bits. Two further knobs, `--sbr-tran-quiet` and `--sbr-tran-dom`, live in the fast transient detector, which runs only for AAC-ELD with SBR (`-p 39 -L 1`; ELD and LD need M4A, LATM or LOAS, not ADTS). `--sbr-tran-quiet` changes ELD+SBR streams at values below 100; for `--sbr-tran-dom` we have not observed a change on our test material.

The **missing harmonics detector** handles the opposite problem. When SBR copies a low band upwards, a tone that existed in the original can land in the wrong place or vanish. The detector notices this and fabricates a synthetic harmonic to put it back. Set it too eagerly and you get whistling and metallic artefacts that were never in the recording; set it too conservatively and bells, glockenspiel and cymbals go dull as their partials quietly disappear.

`--sbr-mh-tone <n>` scales how tonal a band must be before a harmonic is added, and is the main control here. `--sbr-mh-diff <n>` scales the tonality *difference* between the original and the patched band that triggers compensation. `--sbr-mh-decay-orig` and `--sbr-mh-decay-diff` change how long an already-found tone keeps being tracked as it fades, which is what you hear on cymbal and bell tails. `--sbr-mh-sfm-sbr` and `--sbr-mh-sfm-orig` move the spectral-flatness line between "this is a tone" and "this is noise", for the patched and the original band respectively. `--sbr-mh-maxcomp` caps how much envelope compensation is applied around an added harmonic, which affects how much noise sits next to it.

Finally, `--sbr-noise-max <6|3|-3>` sets the ceiling on how loud the noise SBR injects may get — 1.0, 0.5 or 0.125 respectively. This is the straightest control over "air versus hiss" in the top end. It was already a configuration field inside FDK, chosen from the bitrate tuning table and never exposed.

All of these are opt-in: at their neutral values the output is bit-identical to stock, and the test suite enforces that.

17. TNS, PNS, and the afterburner

`--tns-mask <n>` / `--tns-order <n>` — TNS (Temporal Noise Shaping) shapes the quantization noise in time, so it hides under transients (important for sharp attacks, e.g. percussion). The mask is a bitmap of active filters, the order is the length of the filter.

`--pns <0|1>` — PNS (Perceptual Noise Substitution) replaces noise bands with the description "there's noise of such energy here" instead of encoding it precisely. A big saving on noise material.

`--pns-start <Hz>` — from what frequency PNS may act.

`--force-pns` — bypass the low-bitrate gate for PNS. EXPLANATION: FDK has a table that at very low bitrates (below about 28 kbps) completely disables PNS — and then `--pns-start` is ignored, because PNS doesn't work at all. That's why at 24 kbps you heard artifacts like from MP3, and at 64 kbps PNS worked. This flag bypasses the table and turns on PNS despite the low bitrate.

A word on the units before the individual knobs, because it makes the rest make sense. FDK keeps these PNS detection parameters as fixed-point integers internally, and our switches take a plain decimal MULTIPLIER: whatever you type is multiplied by 100 and used to scale FDK's factory value. So `1.0` means "×1.0 = leave the factory value alone", `1.5` means "×1.5", `0.5` means "×0.5", and `-1` means the switch is off entirely. That single convention (`value × 100`, `1.0` = no change) applies to every `--pns-*` scaling knob below.

`--pns-gain <x>` — the LOUDNESS of the fabricated PNS noise, and the one PNS knob you'll reach for most. When PNS decides a band is "just noise", it throws away the actual spectral lines and stores only a single number: the noise energy of that band. On playback the decoder regenerates random noise scaled to that energy. `--pns-gain` scales that stored energy directly. `1.0` = unchanged (the regenerated noise carries the original band's energy); `>1.0` makes the noise fill louder than the original (useful when the substituted noise sounds too timid and the mix goes dull); `<1.0` makes it quieter (when the noise fill hisses too much). Think of it as the "how loud is the substituted noise" dial. Decimal input, `-1` = off.

The remaining PNS knobs govern WHICH bands get turned into noise in the first place — the detection stage — rather than how loud the result is. `--pns-tonality <x>` scales the tonality detection threshold. PNS only fires on bands the encoder judges "noise-like" (low tonality); raising this threshold lets more bands — even somewhat tonal ones — qualify as noise, so the noise substitution gets WIDER (more of the spectrum replaced by cheap noise, more saving but also more risk of smearing genuine tones). Lowering it makes PNS pickier. `1.0` = default.

`--pns-refpower <x>` scales the reference-power detection threshold. PNS compares each band's power against a reference before deciding it is a noise candidate; this knob moves that power gate. `1.0` = default; it interacts with tonality — both must agree for a band to become PNS.

`--pns-gapfill <x>` scales the gap-fill threshold. When PNS has marked bands on either side of a small gap, this heuristic can "close the gap" and mark the band between them as PNS too, so you don't get a coded island stranded between two noise bands. Advanced and subtle — rarely audible on its own. `1.0` = default.

`--pns-min-width <n>` sets the minimum SFB width, in spectral lines, that a band must have before PNS is allowed to act on it. This one is a raw line count, not a `×100` multiplier. It is effective only above the built-in default (LC = 16 lines); pushing it to 32 or 64 restricts PNS to the wider bands only, keeping the encoder from substituting noise on narrow low bands where it would be more obvious. `-1` = off (use FDK's default).

Transients, pre-echo and holes in the core (LC, LD, ELD)

At a low bitrate with a wide band (e.g. LC stereo 32 kbps and `-w 17000`) the core has no bits for every band. You hear it in two ways: noise smeared ahead of a hit (pre-echo) and bubbling, flickering bands that keep disappearing (holes). These knobs reach both mechanisms. Without them the encoder behaves exactly like stock.

HOLES. `--hole-ladder <1..8>` is the strongest knob. When a frame cannot afford everything, FDK zeroes bands in 8 steps, from the quietest upwards. Fewer steps means only the quietest may be zeroed, and the rest of the saving is done by coarser quantisation. Measured at 32 kbps and 17 kHz: zeroed bands drop from 37% to 8% (`2`), at 40 kbps from 37% to 1.7%. In return the remaining bands carry a little more noise, so instead of holes you get a slightly grainier fill. Start with `2`. `--avoid-holes 1` turns on the protection of narrow loud bands that FDK switches off below 40 kbps stereo. `--hole-start <sfb>` protects the lower bands from zeroing (higher = more protected), `--hole-start-short` does the same for short blocks, and `--hole-spread <dB>` fine-tunes when a band counts as covered by its neighbours.

PRE-ECHO. The strongest is `--window-shape 1`: every block, short ones too, gets the KBD window, which leaks less noise far from the sound. Measured: pre-echo 3.4 to 6.1 dB lower on average at 24, 40 and 128 kbps. `--window-shape 0` does the opposite, sine everywhere. `--attack-decay <x>` (stock 0.7) sets the memory of the hit detector: higher compares an attack with a longer average; in the measurement `0.9` lowered the worst pre-echo case by 2.9 dB. Lowering `--attack-floor <dB>` makes quiet attacks such as brushes switch to short blocks too. `--short-groups <0..3>` says how the 8 short windows are grouped: `1` one group (cheapest), `2` even pairs, `3` tight around

the hit. `--preecho-rise <dB>` and `--preecho-depth <dB>` guard pre-echo inside long blocks; in LC they act weakly, because real attacks go to short blocks anyway, but in LD and ELD, which have no short blocks, they are one of the few tools. The other one there is `--tns-thresh <x>`: below 1 TNS switches on more often and shapes the noise in time. `--tns-start <Hz>` sets how low the lower TNS filter reaches.

A start for LC stereo 24 to 50 kbps with a wide band:

```
fdkaac-franken-x64.exe -p 2 -b 32000 -w 17000 --hole-ladder 2 --avoid-holes 1 --window-shape 1 -o out.m4a in.wav
```

`--ath-scale <n>` — scaling of the threshold of hearing (ATH), in Q8 (256 = ×1.0). This is the GLOBAL masking regulator. Above 256 = raises the thresholds = the encoder deems more things inaudible = more aggressive, fewer bits. Below 256 = lowers the thresholds = the encoder preserves more detail = more bits. This is the simplest way to tell the encoder "be cleaner" or "be more economical."

Part V. Details at High Bitrate and Tuning for Speech

18. "More bits = more detail": how to really squeeze it out

A question that often comes up: since I'm giving 400 kbps, will the encoder really use it on a better description of details, or does part get wasted? In AAC there aren't such advanced, separate regulators as in MusePack (signal-to-mask ratio, tone-masks-noise, noise-masks-tone as separate knobs). But the effect "describe more precisely, mask less" we achieve with two tools that together do exactly the same thing.

`--ath-scale <n>` BELOW 256 — lowers the global threshold of hearing. The encoder stops assuming something is inaudible and starts encoding it precisely. This is the closest equivalent of lowering the masking threshold from MusePack. At 320–400 kbps try 200 or 180.

`--spread-mask <n>` BELOW 256 — controls the SPREADING of masking between neighboring bands. In psychoacoustics a loud band "spreads" its masking ability onto its neighbors (that's exactly tone-masks-noise). By reducing this parameter, you tell the encoder: assume less that neighboring bands mask each other — so more bands are treated as audible and precisely encoded. The biggest effect where bits are the limit (96–192 kbps); at very high bitrate the encoder has a surplus of bits anyway, so the change tends to be small.

An audiophile RECIPE (maximum detail, high bitrate, material with a rich top):

```
fdkaac-franken-x64.exe -p 2 -b 400000 --ath-scale 190 --spread-mask
128 -o out.m4a in.wav
```

The combination: a lowered global threshold (ath-scale 190) + reduced spreading of masking (spread-mask 128). The encoder preserves considerably more microdetail, which in a phase comparison with the original gives smaller differences. This is in the spirit of what MusePack did at 1000 kbps — within the bounds of what AAC allows exposing without rewriting the whole model.

For material where you want more of the budget in the stereo width, add `--side-bias 6` — steering bits into the side channel is another place where a high bitrate really gives better sound. (The old `--ms-precision` knob did something similar globally but is very experimental and largely superseded; `--side-bias` is the right tool now.)

Beyond ath-scale there's a more direct lever, and it's worth understanding the units. These min-SNR knobs work on the Q8 fixed-point scale, where 256 stands for 1.0 (so 256 = "leave it alone", 128 = ×0.5, 512 = ×2.0); `-1` means the switch is off. `--minsnr-scale <n>` (Q8, 256 = off) is the closest thing FDK has to MusePack's TMN/NMT knobs: it scales the REQUIRED per-band coding SNR (`sfbMinSnrLdData`), i.e. the minimum signal-to-noise ratio the encoder insists on achieving in each scale-factor band. Values BELOW 256 demand a HIGHER SNR — the encoder has to code each band more accurately, so more detail and more bits — while values above 256 relax the demand and code more coarsely. It's more effective than `--ath-scale` because min-SNR is exactly the floor that FDK's "avoid holes" logic clamps the thresholds back to; touching the threshold copy alone (as ath-scale does) is partly undone downstream, but the min-SNR floor is not.

The two clamp knobs move the hard limits that FDK puts around that required SNR. Internally FDK never lets a band demand more than a MAX_SNR ceiling (about −1 dB) nor less than a MIN_SNR floor (about −25 dB), so even an aggressive `--minsnr-scale` is capped by these. `--minsnr-clamp-hi <n>` (Q8, 256 = off) scales the MAX_SNR ceiling, letting demanding bands ask for MORE than the factory cap — this is what you raise if `--minsnr-scale` alone seems to "run out of room" at the top. `--minsnr-clamp-lo <n>` (Q8, 256 = off) scales the MIN_SNR floor at the other end. Together they widen FDK's factory window so `--minsnr-scale` has somewhere further to push.

Finally, `--reduce-clamp 0` removes the "29 dB Ratio" ceiling on threshold reduction inside the CBR quantizer. When the CBR bit-allocation loop is short on bits it raises (loosens) thresholds, but stock FDK refuses to reduce a threshold by more than about 29 dB relative to the reference — a safety clamp. Setting `--reduce-clamp 0` lifts that clamp, letting the encoder push thresholds deeper and pour bits into the most demanding bands. It pairs

naturally with `--minsnr-scale` for extreme detail, and it only affects CBR (VBR runs a different path). The default is `1` (clamp on, stock behaviour).

19. Tuning for human speech

Yes, in HE-AAC (specifically in the SBR layer) there's a separate tuning mode for speech. In ordinary FDK it's hard-wired as off — here we've exposed it with a flag:

`--speech` — turns on SBR tuning for human speech. It changes the inverse-filtering thresholds, the noise level, and disables parametric encoding of the upper band — all so that speech (podcast, audiobook, dialogue) at low bitrate sounds more natural, without "gurgling" up top. It applies to HE-AAC/HE-AAC v2 (because it works in SBR).

```
fdkaac-franken-x64.exe -p 5 -b 32000 --speech -o mowa.m4a podcast.wav
```

`--speech` changes several things at once. Its SBR estimator part can now be taken separately: `--sbr-invf-table 1` gives just the speech decision table, `--sbr-nf-weight 1.0` just the speech noise weight, and `--sbr-softness 4` both, without changing the crossover and without disabling parametric coding.

Honestly about LC: pure AAC-LC (without SBR) does NOT have a separate, distinct "speech mode" in FDK. The LC psychoacoustic model is the same for speech and music. For speech in LC it's best to simply lower the band (`--core-cutoff` e.g. 12000–14000 Hz, because speech doesn't have much above that) and possibly turn on `--pns` with `--force-pns` at very low bitrates. But there's no dedicated "speech" switch for LC, because it's not in the codec itself — and I don't want to pretend it is.

Part VI. Ready-Made Recipes (From Extreme to Extreme)

20. A set of practical recipes

MUSIC, HIGH ARCHIVAL QUALITY (transparent stereo):

```
fdkaac-franken-x64.exe -p 2 -b 256000 --afterburner 1 -o out.m4a
in.wav
```

MUSIC, "BREATHING" QUASI-VBR (doesn't cut after climaxes):

```
fdkaac-franken-x64.exe -p 2 -b 160000 --peak-bitrate 256000 --vbr-
reservoir 16000 -o out.m4a in.wav
```

AUDIOPHILE, FULL BAND 96 kHz + maximum detail:

```
fdkaac-franken-x64.exe -p 2 -b 400000 --uncap-bandwidth --core-cutoff
40000 --ath-scale 190 --spread-mask 128 -o out.m4a in96k.wav
```

MUSIC STREAMING, MEDIUM BITRATE:

```
fdkaac-franken-x64.exe -p 5 -b 64000 --sbr-stereo-mode 3 -o out.m4a
in.wav
```

PODCAST / SPEECH, LOW BITRATE:

```
fdkaac-franken-x64.exe -p 5 -b 32000 --speech --core-cutoff 12000 -o
out.m4a mowa.wav
```

EXTREMELY LOW (experiment, 8 kbps stereo):

```
fdkaac-franken-x64.exe -p 29 -b 8000 --unlock-bitrate -o out.m4a
in48k.wav
```

RESCUING STEREO AT LOW BITRATE (aggressive intensity):

```
fdkaac-franken-x64.exe -p 2 -b 96000 --is 1 --is-aggression 70 -o
out.m4a in.wav
```

MS ONLY UP TOP (bottom full stereo, top merged):

```
fdkaac-franken-x64.exe -p 2 -b 128000 --msbands-lo 44 --msbands-hi 48
-o out.m4a in.wav
```

21. How to approach your own experiments

Change ONE parameter at a time and listen (or compare the spectrum/phase with the original). The encoder with no Frankenstein flags behaves exactly like the original FDK — that's your reference point. Every flag is a deliberate deviation from the default, proven behavior.

The order worth trying when there are problems:

- "too little detail / too flat" → `--ath-scale` down, then `--spread-mask` down.
- "stereo too narrow up top" → `--sbr-stereo-mode 1` (HE-AAC) or less aggressive IS.
- "metallic highs" → `--sbr-invf 2` or 3.
- "cuts after loud fragments" → `--peak-bitrate` + `--vbr-reservoir` (chapter 8).
- "artifacts like MP3 at very low bitrate" → `--force-pns`.
- "I want the full band from 96 kHz" → `--uncap-bandwidth --core-cutoff 40000`.

22. A closing note on integrity

All the behaviors described here were verified by measurement (stream decodability, real bitrate spread, spectral content), not just assumed. Where something has a limitation (IPD/OPD unsupported, no speech mode in LC, the ceiling of 6144 bits per channel, a residual bitrate floor of ~10 kbps), it is said outright, instead of promising things the codec can't do.

The encoder without flags = pure FDK. The flags = your deliberate decisions. Happy tuning.

Part VII. Reference Tables

Three orientation tables computed from the tuning tables in the FDK code. They are APPROXIMATE (the band grid is stepped), but they show the right order of magnitude and help you deliberately choose settings without guessing.

23. Table 1 — the upper frequency of the band (SFB)

The spectrum is divided into bands (scale factor bands, SFB), numbered from the bottom: band 0 is the lowest bass, the higher the number, the higher in frequency. When you restrict something to "N bands" (`--msbands`, `--isbands`), this table tells you what frequency that roughly corresponds to. Every fourth band is shown; the last row is the total number of bands and the Nyquist frequency (half the sample rate).

SFB	16 kHz	22.05 kHz	32 kHz	44.1 kHz	48 kHz	96 kHz
0	62	43	62	86	94	188
4	312	215	312	431	469	938
8	562	388	562	775	844	1688
12	875	646	1000	1378	1500	2438
16	1250	991	1500	2067	2250	3750
20	1656	1335	2250	3101	3375	5625
24	2188	1852	3375	4651	5062	8062
28	2875	2584	5000	6891	7500	12938
32	3844	3618	7000	9647	10500	24000
36	5188	5039	9000	12403	13500	36000
40	7000	7020	11000	15159	16500	48000
44	—	9647	13000	17916	19500	—
48	—	—	15000	22050	24000	—
numbe	43 /	47 /	51 /	49 /	49 /	41 /

SFB	16 kHz	22.05 kHz	32 kHz	44.1 kHz	48 kHz	96 kHz
r of bands / Nyquist	8000	11025	16000	22050	24000	48000

An example of how to read it: at 44.1 kHz band no. 40 ends around 15.2 kHz. If you want MS to work only below ~15 kHz, set `--msbands 40`.

24. Table 2 — SBR: the start index vs. the crossover frequency

In HE-AAC the lower part of the band is encoded by the AAC core, and the upper part is added by SBR. `--sbr-start` (index 0..15) decides at what frequency SBR begins — a lower index means SBR comes in lower, so the core is narrower. "core" is the core's sample rate; in dual-rate mode the output is twice as high as the core.

start index	core 16 kHz	core 24 kHz	core 32 kHz	core 44.1 kHz	core 48 kHz
0	2750	2250	2500	1378	1500
1	3000	2625	3000	2067	2250
2	3250	3000	3500	2756	3000
3	3500	3375	4000	3445	3750
4	3750	3750	4500	4134	4500
5	4000	4125	5000	4823	5250
6	4250	4500	5500	5512	6000
7	4500	4875	6000	6202	6750
8	4750	5250	6500	6891	7500

The upper limit of SBR is set by `--sbr-stop` (0..13) — a higher index means SBR reaching higher. By default the library picks both indices for the bitrate; give these values only when you deliberately want to shift the crossover point.

25. Table 3 — AAC-LC: automatic band cutoff by bitrate

When you don't give `-w`, the encoder itself picks the bandwidth from this table — by bitrate PER CHANNEL (128 kbps stereo is 64 kbps per channel). The table is common for 32/44.1/48 kHz and higher; the sample rate affects only the upper limit (Nyquist).

bitrate per channel	mono band	stereo and more band
up to 12 kbps	3700	5000
20 kbps	6900	9640
28 kbps	9600	13050
40 kbps	12060	14260
56 kbps	13950	15500
72 kbps	14200	16120
96 kbps and more	17000	17000

A practical conclusion: in pure AAC-LC (without SBR) the automation stops around 17 kHz anyway. Giving `-w` higher makes sense only with a large bitrate reserve; a full band above 20 kHz requires `--uncap-bandwidth` and a sample rate of at least 96 kHz.

26. How to read `--verbose`

`--verbose` prints raw values without hints in parentheses, so the readout is clean. Here's what the less obvious ones mean:

- AOT (profile): 2 = AAC-LC, 5 = HE-AAC, 29 = HE-AAC v2, 23 = AAC-LD, 39 = AAC-ELD.
- bitrate-mode: 0 = CBR, 1 to 5 = VBR (higher = better quality).
- channel-mode: 1 = mono, 2 = stereo. In HE-AAC v2 the core is mono, and stereo is recreated from parameters (Parametric Stereo).
- core bandwidth: the upper frequency of the AAC core. The SOURCE is given in parentheses: "from -w" (you gave it manually), "from --core-cutoff" or "auto". Under SBR this is only the core — SBR plays above this value.
- final BW (AAC+SBR): shown only with SBR active — the approximate upper frequency of the WHOLE signal (core plus SBR), computed from the SBR stop index. It's a complement to "core bandwidth": that says how far the core alone reaches, and this how far the whole HE-AAC reaches after adding SBR.
- sbr-ratio: 1 = single-rate (downsampled), 2 = dual-rate (the core at half the output frequency).
- sbr amp res: 0 = resolution 1.5 dB, 1 = 3.0 dB.
- codec delay: the codec delay in samples per channel — important for gapless.
- IS corr threshold, IS L/R ratio: thresholds on the Q8 scale, where 256 = 1.0. A lower correlation threshold means intensity stereo turns on MORE READILY (contrary to intuition).
- franken overrides applied: a list of the switches that in this run deviate from pure FDK (or "none," when you changed nothing).

Values on the Q8 scale (like the IS thresholds, `--ms-precision`, `--ms-bias`, `--ath-scale`, `--spread-mask`) are numbers where 256 means 1.0 — for example 243 is about 0.95.

Part VIII. DAB+ Digital Radio

27. DAB+ output (`--dab`, `--dab-label`)

So far we've been making files — MP4/M4A for players, ADTS for streaming. DAB+ is something else: it's the format of terrestrial digital radio (the standard is ETSI TS 102 563), and this encoder can produce a ready DAB+ stream directly, without any external re-packing.

Why can't you just take an ordinary AAC file and send it over the air? Because DAB+ carries AAC in a way of its own. First, the transform is shorter — 960 samples instead of the usual 1024 — so the frame length fits the radio timing grid. Second, the audio is packed into a **super frame** that lasts exactly 120 milliseconds. Third, and most importantly, the air is a hostile place: the signal fades, cars drive through tunnels, interference comes and goes. So DAB+ wraps the sound in two layers of protection: a **firecode** (a Fire CRC) guarding the header, and **Reed-Solomon** error-correction coding — specifically RS(120,110) over the Galois field GF(256) — which lets the receiver rebuild data even when part of the signal arrives damaged. The encoder does all of this for you.

What comes out is a raw `.dabp` stream: one super frame after another, nothing else. This is not a file you play in a media player — it's the raw material a DAB+ multiplexer eats. In practice the chain looks like this:

```
out.dabp → odr-dabmux → ETI → transmitter (or a software receiver: welle.io, dablin)
```

`odr-dabmux` is the multiplexer that combines several stations into one ensemble and produces an ETI stream, which then goes either to a real transmitter or to a soft receiver for testing.

Turning it on: `--dab`

```
fdkaac --dab -b 96 -o out.dabp input.wav
```

The `--dab` switch flips the encoder into DAB+ output mode. A few rules the standard forces on you, and the encoder enforces them:

- The sample rate MUST be 32000 or 48000 Hz. Nothing else is legal in DAB+.
- The bitrate must be a multiple of 8 kbps, from 8 up to 192 kbps.
- Mono and stereo are both fine.

The interesting part is that you usually DON'T pick the profile yourself. DAB+ practice (and the reference tool `odr-audioenc`) is to derive the AAC profile from the bitrate and the channel count, so that low bitrates automatically use the more economical tools:

- stereo at 48 kbps or below → **HE-AAC v2** (with Parametric Stereo),
- mono up to 64k, or stereo up to 80k → **HE-AAC** (with SBR),
- above that → **AAC-LC** (the plain full-band profile).

So `-b 96` on a stereo file lands on AAC-LC, `-b 64` on HE-AAC, and `-b 32` on HE-AAC v2 — you don't have to think about it. If you insist, you can still force the profile with `-p` (2 = LC, 5 = HE-AAC, 29 = HE-AAC v2), the same as in every other mode of this encoder.

```
fdkaac --dab -b 64 -o out.dabp input.wav    # → HE-AAC automatically
fdkaac --dab -b 32 -o out.dabp input.wav    # → HE-AAC v2
automatically
```

The station label: `--dab-label`

A DAB+ receiver shows text on its display — the station name, and often the title of what's playing. In DAB+ that text travels in a channel called DLS (Dynamic Label Segment), tucked as X-PAD data inside the super frame. This encoder can carry a STATIC label — one fixed string for the whole file:

```
fdkaac --dab -b 96 --dab-label "Radio DHT" -o out.dabp input.wav
```

Up to about 48 characters fit (three segments packed into one PAD). "Static" is the key word: the text does not change over the course of the file. A truly dynamic label that updates track by track, and the MOT slideshow that puts album art on the screen (the job of `ODR-PadEnc` in the OpenDigitalRadio world), are on the roadmap but not here yet. And note: `--dab-label` without `--dab` does nothing — the label is simply ignored, because there is no super frame to put it in.

A word on trust

Everything above was checked against independent tools, not just "it compiled." The streams decode cleanly in faad2 (the decoder behind dablin), the DLS label reads back correctly on the receiver side, and the AAC-LC output is bit-identical to what the reference `odr-audioenc` produces from the same input. Nine combinations were tested — 48 and 32 kHz, mono and stereo, across all three profiles — and each one decodes on its own. As everywhere in this encoder: if you don't pass `--dab`, nothing changes, and the ordinary file output stays bit-identical to stock FDK.

Part IX. Glossary

For quick recall — all the terms from this manual, explained in one or two sentences, in the language of the audio engineer.

AAC (Advanced Audio Coding). A family of lossy audio codecs from the MPEG family, the successor to MP3. Better quality per bit than MP3, especially at low bitrates.

AAC-LC (Low Complexity). The basic AAC profile. Encodes the full band classically. The choice for high bitrates and archiving.

Afterburner. An additional, slower algorithm optimizing quantization in FDK. On, it gives slightly better quality. On by default and recommended.

ATH (Absolute Threshold of Hearing). The threshold of hearing — the limit of silence dependent on frequency, below which the ear doesn't register sound. The encoder throws away what is below it. In this encoder regulated by `--ath-scale`.

Bitrate. The amount of data per second of sound, in kilobits (kbps) or bits (bps). Higher = more room for details = better quality (up to a certain limit).

Bit reservoir. A buffer allowing bits to be borrowed between frames. Thanks to it even CBR locally "breathes." The basis of our quasi-VBR.

CBR (Constant Bitrate). A fixed bitrate. Predictable size, convenient for streaming. In AAC slightly variable anyway thanks to the reservoir.

Coupling (SBR). A stereo mode in the SBR layer in which both channels share a common envelope plus a level description — economical, but a narrower scene in the upper band.

Cutoff. The upper limit of the band encoded by the core. Above it either silence, or (in HE-AAC) SBR takes over the job.

HE-AAC (High Efficiency). AAC-LC + SBR. The core encodes the bottom of the band, SBR recreates the top. For medium and low bitrates.

HE-AAC v2. HE-AAC + Parametric Stereo. For the lowest stereo bitrates.

ICC (Inter-channel Coherence). A PS parameter describing how similar/coherent the channels are to each other. Controlled by `--ps-icc`.

IID (Inter-channel Intensity Difference). A PS parameter describing the loudness difference between channels. The primary carrier of the stereo image in PS.

Intensity Stereo (IS). A technique in which high bands are encoded as one energy envelope instead of two channels. Economical, but can narrow the scene. Controlled by the `--is-aggression` slider.

Inverse filtering. An SBR mechanism "whitening" the copied material of the upper band, controlled by a tonality estimator. Stronger = less metallicness. Regulated by `--sbr-invf`.

IPD/OPD (phase differences in PS). NOT supported in the FDK encoder (always zero).

Quantization. Rounding spectrum coefficients to a finite precision. The fewer the bits, the larger the error (noise/holes). The heart of lossy compression.

Masking. The phenomenon in which a loud sound makes you not hear quieter ones right next to it (in the spectrum or in time). The main source of bit saving in AAC.

MDCT. A transform carrying a frame of sound from time to the frequency domain. The whole psychoacoustic model works on its result.

MS Stereo (Mid/Side). Encoding the sum (mid) and difference (side) instead of L and R. Almost free saving when the channels are similar.

PNS (Perceptual Noise Substitution). Replaces noise bands with the description "there's noise of such energy here" instead of encoding them precisely. Controlled by `--pns`, `--force-pns`.

Parametric Stereo (PS). Encodes one channel + parameters (IID, ICC), from which the decoder recreates stereo. For the lowest bitrates.

Profile (AOT, Audio Object Type). The AAC variant selected by `-p` (LC, HE, v2, LD, ELD).

Frame. A portion of sound (1024 samples) that the encoder processes at once.

SBR (Spectral Band Replication). Reconstruction of the upper band from the lower plus a small description. The heart of HE-AAC.

SFB (Scale Factor Band). A scale factor band. A group of neighboring spectrum coefficients for which the encoder makes a common decision about precision. Numbered from the bottom (0 = bass).

Spreading (of masking). The propagation of masking ability onto neighboring bands. Reduced by `--spread-mask` for more detail.

Tuning table. A set of default SBR/PS settings chosen by FDK by bitrate and frequency. Our flags override it.

TNS (Temporal Noise Shaping). Shapes the quantization noise in time, so it hides under transients. Important for percussion and sharp attacks.

Transient. A sharp, short sound (a hit, an attack). Difficult for the codec, because the quantization noise can "outrun" the attack (pre-echo).

VBR (Variable Bitrate). A variable bitrate dependent on the difficulty of the material. In FDK weak; our quasi-VBR (chapter 8) is a better alternative.

End of glossary.